



Ashim Dahal

USING THE CYBER INNOVATION CLUSTER

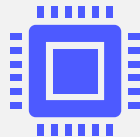
INTRODUCTION TO DISTRIBUTED COMPUTING



A network of interconnected computers that work together to solve one problem.



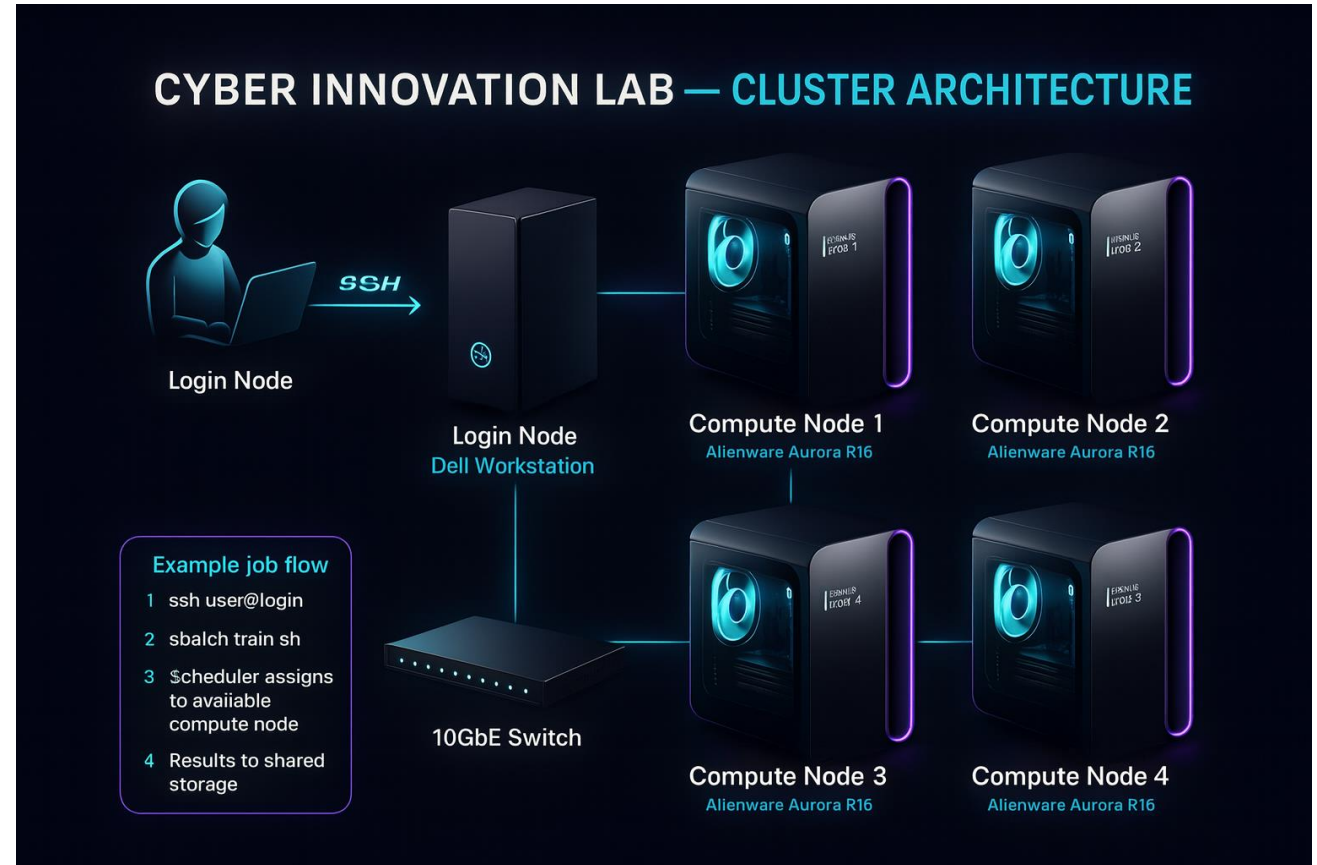
A cluster of 4 computers does not imply the 4 computers would appear to be one single computer



We chunk our input data into smaller pieces or divide the problem into sub problems such that each computer can work on their part of the solution individually.

THE CYBER INNOVATION CLUSTER

- 1 x login node
- 4 x compute node each with RTX 5070 ti and 20 CPU cores
- Passwordless SSH between devices
- 30 x faster than kaggle
- 60-90 x faster than google colab

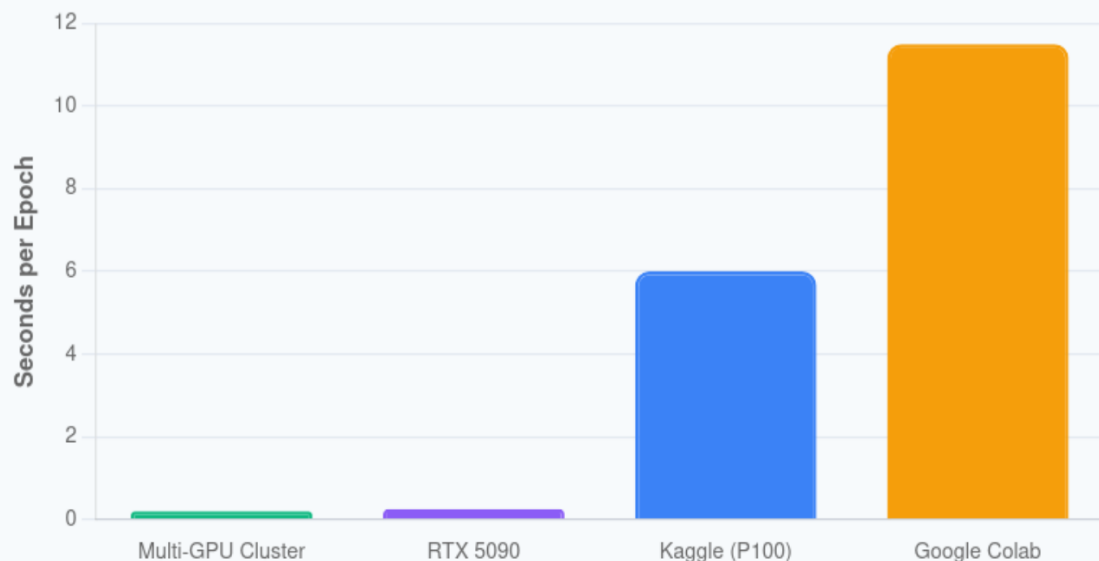




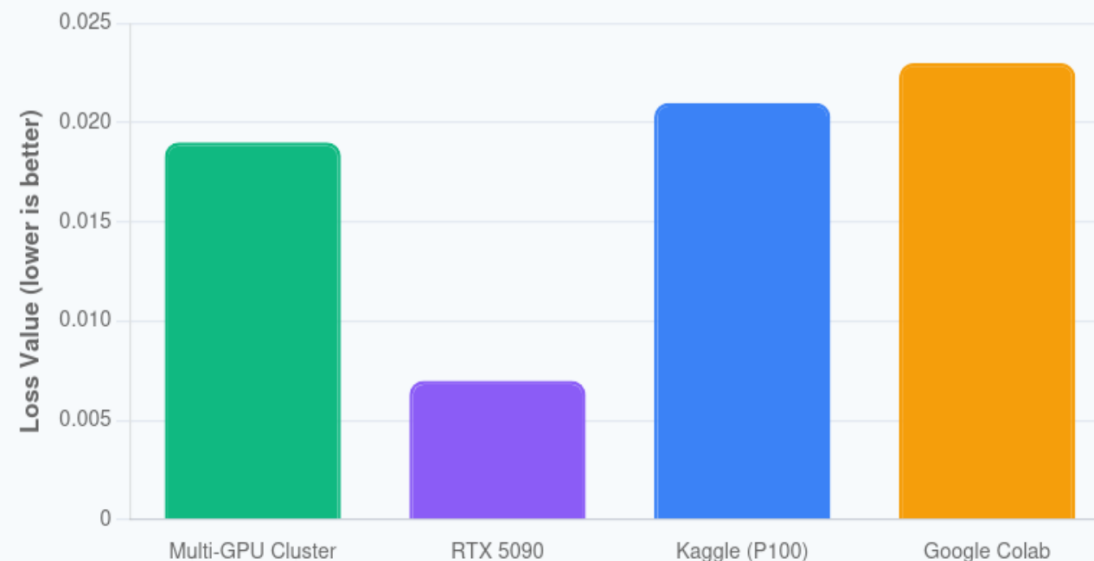
Distributed Training Performance Comparison

Average Epoch Duration & Loss Analysis Across Different Platforms

Average Epoch Duration (seconds)



Final Average Loss



Multi-GPU Cluster (4x GPU)

0.2s

Loss: 0.019

RTX 5090 (Single GPU)

0.26s

Loss: 0.007 ★

Kaggle (Tesla P100)

6.0s

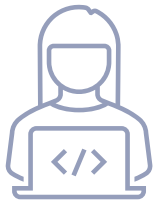
Loss: 0.021

Google Colab

11.5s

Loss: 0.023

PLEASE ABSOLUTELY DO NOT



Run your code in Login
Node



Install tools without
checking modules



Make conda
environment outside
your home folder



Occupy more resources
than needed by your
models

HOW TO RUN PYTORCH CODE?



Step 0: load miniconda module with

```
module load miniconda/miniconda3 # see list of available modules with  
module avail
```



You should now have conda environment.



Usually, you'd create conda environment with

```
conda create -n <env_name> python=3.12 # DONOT DO THIS IN THE  
LOGIN NODE
```



In the login node you need to store this environment within your home folder



In linux the home folder for your user is denoted by the "~" sign:

```
conda create -p ~/<env_name> python=3.12 # you may choose any other  
path but having it on the home folder makes it easier to load in future
```

HOW TO RUN PYTORCH CODE?

- Activate your virtual environment
 - `conda activate ~/<env_name>`
- Our machines require cuda 12.8+ to run without hiccups so when installing torch make sure to select minimum 12.8 version of cuda
 - `pip3 install torch torchvision` #the default version as of today is 12.8
- Now we are ready to modify our pytorch code into Distributed Data Parallel (DDP) code

EXAMPLE

- A basic CNN model in pytorch without DDP
- Define the CNN model w necessary modules imported

```
1 #!/usr/bin/env python3
2 import os, time, warnings
3 from datetime import datetime
4 import torch, torch.nn as nn, torch.optim as optim
5 from torch.utils.data import DataLoader
6 from torchvision import datasets, transforms
7
8 warnings.filterwarnings("ignore")
9 torch.backends.cudnn.benchmark = True
10 torch.backends.cuda.matmul.allow_tf32 = True
11 if hasattr(torch, "set_float32_matmul_precision"):
12     torch.set_float32_matmul_precision("high")
13
14 class SimpleCNN(nn.Module):
15     def __init__(self):
16         super().__init__()
17         self.net = nn.Sequential(
18             nn.Conv2d(1, 32, 3, 1, 1), nn.ReLU(),
19             nn.MaxPool2d(2),
20             nn.Conv2d(32, 64, 3, 1, 0), nn.ReLU(),
21             nn.MaxPool2d(2),
22             nn.Flatten(),
23             nn.Linear(64*6*6, 128), nn.ReLU(),
24             nn.Linear(128, 10),
25         )
26     def forward(self, x): return self.net(x)
27
```



```

28 def main(epochs=3, batch_size=128):
29     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
30     transform = transforms.Compose([
31         transforms.ToTensor(),
32         transforms.Normalize((0.1307,), (0.3081,))
33     ])
34     dataset = datasets.MNIST("./data", train=True, transform=transform, download=True)
35     num_workers = max(1, (os.cpu_count() or 2) - 1)
36     loader = DataLoader(dataset, batch_size=batch_size, shuffle=True,
37                         num_workers=num_workers, pin_memory=(device.type=="cuda"),
38                         prefetch_factor=4 if num_workers > 1 else 2, persistent_workers=True)
39
40     model = SimpleCNN().to(device)
41     opt = optim.Adam(model.parameters(), lr=1e-3)
42     loss_fn = nn.CrossEntropyLoss()
43
44     for epoch in range(epochs):
45         t0, loss_sum, n = time.time(), 0.0, 0
46         model.train()
47         for x, y in loader:
48             x, y = x.to(device, non_blocking=True), y.to(device, non_blocking=True)
49             opt.zero_grad(set_to_none=True)
50             loss = loss_fn(model(x), y)
51             loss.backward()
52             opt.step()
53             loss_sum += loss.item(); n += 1
54         dt = time.time() - t0
55         print(f"[{datetime.now()}] epoch {epoch} duration={dt:.2f}s avg_loss={loss_sum/max(1,n):.4f}")
56
57 if __name__ == "__main__":
58     main()

```

DDP VERSION

```
1 #!/usr/bin/env python3
2 import os, time, warnings, socket
3 from datetime import datetime
4 import torch, torch.nn as nn, torch.optim as optim
5 from torch.utils.data import DataLoader, distributed as dist_data
6 from torchvision import datasets, transforms
7 import torch.distributed as dist
8 from torch.nn.parallel import DistributedDataParallel as DDP
9
10 warnings.filterwarnings("ignore")
11 for k in ("NCCL_DEBUG", "TORCH_DISTRIBUTED_DEBUG"): os.environ.pop(k, None)
12 os.environ.setdefault("PYTHONWARNINGS", "ignore")
13 torch.backends.cudnn.benchmark = True
14 torch.backends.cuda.matmul.allow_tf32 = True
15 if hasattr(torch, "set_float32_matmul_precision"):
16     torch.set_float32_matmul_precision("high")
17
```

DDP SPECIFIC CODE

- Don't worry about this, all these variables would be set up by the SLURM script

```
18 def map_env_from_slurm():
19     if "RANK" not in os.environ and "SLURM_PROCID" in os.environ:
20         os.environ["RANK"] = os.environ["SLURM_PROCID"]
21         os.environ["WORLD_SIZE"] = os.environ["SLURM_NTASKS"]
22         os.environ["LOCAL_RANK"] = os.environ.get("SLURM_LOCALID", "0")
23     os.environ.setdefault("MASTER_ADDR", os.environ.get("SLURM_LAUNCH_NODE_IPADDR", "127.0.0.1"))
24     os.environ.setdefault("MASTER_PORT", "29500")
25
```

ALMOST THE SAME MODEL

- The only difference is the `cpu_workers()` function.
- When submitting a job to SLURM workload manager, we have to request the # of CPUs we would be using for our code which we can retrieve as the OS environment variable

```
26 class SimpleCNN(nn.Module):
27     def __init__(self):
28         super().__init__()
29         self.net = nn.Sequential(
30             nn.Conv2d(1,32,3,1,1), nn.ReLU(),
31             nn.MaxPool2d(2),
32             nn.Conv2d(32,64,3,1,0), nn.ReLU(),
33             nn.MaxPool2d(2),
34             nn.Flatten(),
35             nn.Linear(64*6*6,128), nn.ReLU(),
36             nn.Linear(128,10),
37         )
38     def forward(self,x): return self.net(x)
39
40 def cpu_workers(): return max(1, (int(os.environ.get("SLURM_CPUS_PER_TASK", os.cpu_count() or 2)) - 1))
41
```

THE MAIN DEAL

```
42 def main(epochs=3, batch_size=128):
43     map_env_from_slurm()
44     dist.init_process_group(backend="nccl", init_method="env://")
45     local_rank = int(os.environ.get("LOCAL_RANK", "0"))
46     torch.cuda.set_device(local_rank)
47     device = torch.device("cuda", local_rank)
48
49     rank, world = dist.get_rank(), dist.get_world_size()
50     if rank == 0:
51         print(f"[{datetime.now()}] world_size={world} device={torch.cuda.get_device_name(device)}")
52
53     transform = transforms.Compose([transforms.ToTensor(),
54                                     transforms.Normalize((0.1307,), (0.3081,))])
55     if rank == 0:
56         _ = datasets.MNIST("./data", train=True, transform=transform, download=True)
57     dist.barrier()
58     dataset = datasets.MNIST("./data", train=True, transform=transform, download=False)
59
60     sampler = dist_data.DistributedSampler(dataset, shuffle=True)
61     nw = cpu_workers()
62     loader = DataLoader(dataset, batch_size=batch_size, sampler=sampler,
63                         num_workers=nw, pin_memory=True,
64                         prefetch_factor=(6 if nw>2 else 2), persistent_workers=True)
65
```

THE DDP TRAINING

```
65
66 model = SimpleCNN().to(device)
67 model = DDP(model, device_ids=[local_rank], output_device=local_rank, gradient_as_bucket_view=True)
68 opt = optim.Adam(model.parameters(), lr=1e-3)
69 loss_fn = nn.CrossEntropyLoss()
70
71 for epoch in range(epochs):
72     sampler.set_epoch(epoch)
73     t0, loss_sum, n = time.time(), 0.0, 0
74     model.train()
75     for x, y in loader:
76         x, y = x.to(device, non_blocking=True), y.to(device, non_blocking=True)
77         opt.zero_grad(set_to_none=True)
78         loss = loss_fn(model(x), y)
79         loss.backward()
80         opt.step()
81         loss_sum += loss.item(); n += 1
82     if rank == 0:
83         dt = time.time() - t0
84         print(f"[{datetime.now()}] epoch {epoch} duration={dt:.2f}s avg_loss={loss_sum/max(1,n):.4f}")
85
86     dist.destroy_process_group()
87
88 if __name__ == "__main__":
89     main()
```

HOW TO RUN?

- In order to run distributed code we need to submit our code as a job to SLURM workload manager
- SLURM workload manager would take a look at our requirements and see if those resources are available
- If the resources are available, our code will start to run else slurm will keep them in queue until the resources are available.
- Let's take a look at how to submit a job via slurm.

SLURM SCRIPT DEMO

- Slurm would check if 2 devices with 18 cpus is available and if so it would run the code on those devices.
- We do not have gres gpu and gpu per node flags set up because all our nodes are already gpu and theres at most 1 gpu per node.

```
1 #!/bin/bash
2 #SBATCH --job-name=train_ddp
3 #SBATCH --output=ddp_train.%j.out
4 #SBATCH --error=ddp_train.%j.err
5 #SBATCH --partition=main
6 #SBATCH --nodes=2 # the number of computers you need to work in parallel we have up to 4 but use
7 #SBATCH --ntasks-per-node=1 # one torchrun per node
8 #SBATCH --cpus-per-task=10 # you can allocate up to 18, leave the rest 2 cores for communication
9 #SBATCH --time=02:00:00
10
11 set -euo pipefail
12 #there are other settings as well (used in magnolia) which we don't use. This is a simpler broil
13 # all cluster like ours
14 MASTER_ADDR=$(scontrol show hostnames "$SLURM_JOB_NODELIST" | head -n1)
15 MASTER_PORT=29500
16
17 export NCCL_DEBUG=INFO
18 export NCCL_IB_DISABLE=1
19 export NCCL_SOCKET_IFNAME=eno1,eth0,enp*,ens* # set to your real iface if needed
20 export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
21
22 echo "Nodes: $SLURM_JOB_NODELIST"
23 echo "Master: $MASTER_ADDR:$MASTER_PORT"
24
25 srun bash -lc '
26   echo "[$(hostname)] CUDA_VISIBLE_DEVICES=$CUDA_VISIBLE_DEVICES"
27   torchrun \
28     --nnodes="$SLURM_NNODES" \
29     --node_rank=$SLURM_NODEID \
30     --nproc_per_node=1 \
31     --master_addr="$MASTER_ADDR" \
32     --master_port="$MASTER_PORT" \
33     train_ddp.py
34 '
```

HELPFUL SLURM COMMANDS

- `sbatch train.sh $(submit your request train.sh to the workload manager)`
- `squeue #` shows current jobs in line or being executed in parallel
- `scancel <job_id>` or `scancel -u username`
- Basic linux commands like: `man`, `cat`, `ls`, `la`, etc. Would come very handy.